WEEK 1 · LESSON 2 OF 24 · HANDOUT

Preparing Data: Encoding and Scaling

Study time: **about 50 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=irHhDMbw3xo — open the lesson online to watch inside the course.

🎓 **Go deeper:** Data School: How do I create dummy variables in pandas? — https://www.youtube.com/watch?v=0s_1IsROgDc

🎓 **Go deeper:** SciPy 2018 tutorial: Machine Learning with scikit-learn, Part 1 (scikit-learn core developers) — https://www.youtube.com/watch?v=4PXAztQtoTg

Your goal in this lesson: Turn text categories into numbers with one-hot encoding, put numbers on the same scale with standardisation, and avoid the most common data-preparation mistake: leakage.

Models only understand numbers

A model does maths. It cannot multiply "Gasabo" by anything. Before training, every feature must be a **number**, and many models also need the numbers to be on a **similar scale**. This preparation is often most of the work in a real project.

Run the homes cell again:

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(7)
n = 600
district = rng.choice(["Gasabo", "Kicukiro", "Nyarugenge"], n, p=[0.45, 0.35, 0.2])
bedrooms = rng.integers(1, 5, n)
size_m2 = (bedrooms * 28 + rng.normal(20, 12, n)).clip(25).round()
km_to_centre = rng.uniform(0.5, 15, n).round(1)
furnished = rng.random(n) < 0.3
base = {"Gasabo": 120_000, "Kicukiro": 90_000, "Nyarugenge": 100_000}
rent = (np.array([base[d] for d in district]) + 2_600 * size_m2 - 6_000 * km_to_centre
        + np.where(furnished, 60_000, 0) + rng.normal(0, 25_000, n)).round(-3)
homes = pd.DataFrame({"district": district, "bedrooms": bedrooms, "size_m2": size_m2,
                      "km_to_centre": km_to_centre, "furnished": furnished, "rent_rwf": rent})
print(homes.shape)
homes.head()
```

```
print(homes.dtypes)
print(homes["district"].value_counts())
```

`district` is text and `furnished` is True/False. Both need converting.

True / False → 1 / 0

```
homes["furnished"] = homes["furnished"].astype(int)
print(homes["furnished"].head())
```

Categories → one-hot encoding

Could we write Gasabo = 1, Kicukiro = 2, Nyarugenge = 3? **No.** The model would think Nyarugenge is "three times" Gasabo and that Kicukiro sits "between" them. Districts have **no order**.

Instead we use **one-hot encoding**: one new 0/1 column per category.

district	district_Gasabo	district_Kicukiro	district_Nyarugenge
Gasabo	1	0	0
Kicukiro	0	1	0
Nyarugenge	0	0	1

```
X = pd.get_dummies(homes.drop(columns="rent_rwf"), columns=["district"], dtype=int)
print(X.columns.tolist())
X.head()
```

You now have 7 feature columns, all numbers.

When order is real, keep the order: education level (primary < secondary < university) or clothing size (S < M < L) can be coded 1, 2, 3. This is called **ordinal encoding**.

The scikit-learn way: OneHotEncoder

`pd.get_dummies` is quick for exploring. In real projects we use scikit-learn's `OneHotEncoder`, because it **remembers** the categories it saw in training and handles new ones safely:

```
from sklearn.preprocessing import OneHotEncoder

enc = OneHotEncoder(handle_unknown="ignore", sparse_output=False)
enc.fit(homes[["district"]])
print(enc.categories_)
print(enc.transform(pd.DataFrame({"district": ["Kicukiro", "Musanze"]})))
```

"Musanze" never appeared in the training data, so it becomes all zeros instead of crashing your app.

Scaling: putting numbers on the same ruler

Look at the ranges:

```
print(homes[["bedrooms", "size_m2", "km_to_centre"]].describe().loc[["min", "max"]])
```

Bedrooms go from 1 to 4, size from 25 to about 166. Models that measure **distances** between examples (k-nearest neighbours, Lesson 5) or use gradient descent would treat size as far more important just because its numbers are bigger.

Standardisation rescales each column to mean 0 and standard deviation 1 — the z-score you met in Math & Python for AI:

> *z = (value − mean) ÷ standard deviation*

```
from sklearn.preprocessing import StandardScaler

nums = homes[["bedrooms", "size_m2", "km_to_centre"]]
scaler = StandardScaler()
scaled = scaler.fit_transform(nums)
print(scaled.mean(axis=0).round(2), scaled.std(axis=0).round(2))
```

Every column now has mean 0 and SD 1. `MinMaxScaler` is another option: it squeezes each column into 0–1.

Model	Needs scaling?
Linear and logistic regression	Helps, and matters with regularisation
k-nearest neighbours, SVM	**Yes** — they use distances
Decision trees, random forests	No — they only compare values

The golden rule: fit on training data only

The scaler learns a mean and SD; the encoder learns the categories. They must learn them from the **training set only**, then apply the same numbers to the test set:

```
from sklearn.model_selection import train_test_split

X_train, X_test = train_test_split(X, test_size=0.2, random_state=42)
cols = ["bedrooms", "size_m2", "km_to_centre"]
scaler = StandardScaler().fit(X_train[cols])      # learn from training rows
X_train_s = scaler.transform(X_train[cols])
X_test_s = scaler.transform(X_test[cols])         # reuse the training numbers
print(X_test_s.mean(axis=0).round(2))             # close to 0, not exactly 0
```

If you fit the scaler on all the data, information about the test set **leaks** into training and your test score becomes too optimistic. This mistake is called **data leakage**. In Week 4 you will use a **Pipeline**, which makes this rule automatic.

Missing values

Real data has gaps. Two simple choices: drop the rows (if very few) or **impute** — fill them with the column's median (numbers) or most common value (categories). scikit-learn's `SimpleImputer(strategy="median")` does this, and like the scaler it must be fitted on training data only.

Key terms

- **One-hot encoding** — one 0/1 column per category, for categories with no order.
- **Ordinal encoding** — numbers 1, 2, 3 for categories that have a real order.
- **Standardisation** — rescale to mean 0, SD 1 (StandardScaler).
- **Imputation** — filling missing values.
- **Data leakage** — test-set information sneaking into training.

Remember this: Convert every feature to numbers, scale them when the model uses distances, and always learn preparation steps from the training data only.

Name: _____ Date: _____

Activity

Task 1

One-hot encode `district` with `drop_first=True`. How many district columns do you get, and why is that still enough information?

Task 2

Scale `size_m2` with `MinMaxScaler`. What are the new minimum and maximum?

Task 3

Make a small DataFrame with a column `education` (primary, secondary, university) and code it in order 1, 2, 3 with `.map()`.

Task 4

Put three missing values into `size_m2` and fill them with `SimpleImputer(strategy="median")`.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. Why can't we code Gasabo = 1, Kicukiro = 2, Nyarugenge = 3?
   - A. Python cannot store numbers
   - B. It uses too much memory
   - C. scikit-learn forbids it
   - D. The model would think the districts have an order and size
2. How many 0/1 columns does one-hot encoding make for a feature with 3 categories (without drop_first)?
   - A. 1
   - B. 3
   - C. 2
   - D. 6
3. What does StandardScaler do to each column?
   - A. Sorts it
   - B. Rescales it to mean 0 and standard deviation 1
   - C. Rounds it
   - D. Deletes outliers
4. Which model does NOT need scaled features?
   - A. Decision tree
   - B. k-nearest neighbours
   - C. Logistic regression with regularisation
   - D. Support vector machine
5. Why must the scaler be fitted on the training set only?
   - A. It is faster
   - B. Because fit only works once
   - C. To avoid leaking test-set information into training
   - D. The test set has no numbers
6. What does OneHotEncoder(handle_unknown='ignore') do with a district it never saw?
   - A. Guesses the closest district
   - B. Crashes
   - C. Turns it into all zeros
   - D. Deletes the row

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.